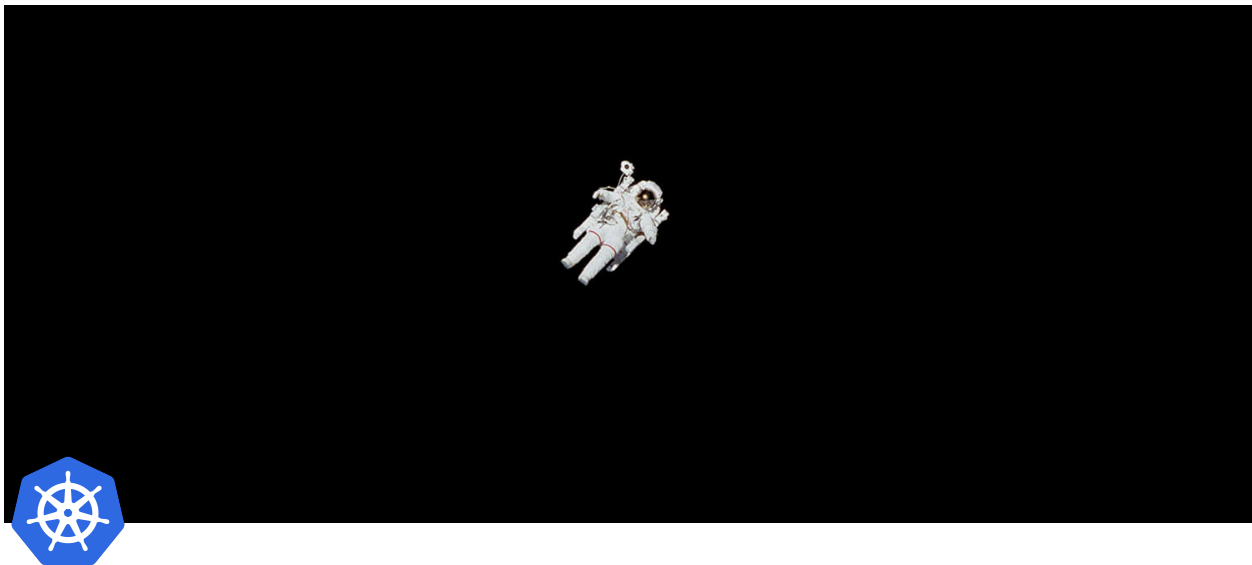




YAML & Kubernetes Lab

Kubernetes Manifest & Resources

A Kubernetes manifest is a YAML file that describes a resource and its desired state.

A resource is any object managed by Kubernetes, such as a Pod, Deployment, or Service, that is created and controlled based on what is defined in the manifest.

Lab Objectives

By the end of this lab, you will be able to:

- Understand what a Kubernetes manifest represents
 - Identify the core structure of a Kubernetes YAML file
 - Write a basic manifest using the declarative model
 - Define and modify Kubernetes resources using YAML
-

Prerequisites

Before starting this lab, you should:

- Have a basic understanding of Kubernetes concepts

- Be familiar with YAML syntax (indentation, key-value pairs)
 - Know what Pods, Deployments, and Services are at a high level
-

Lab Scope

In this lab, you will:

- Write YAML manifests from scratch
- Focus on structure and intent, not tooling
- Practice thinking declaratively rather than imperatively

This lab focuses on **understanding**, not production readiness.

What You Will NOT Do

- No Helm charts
 - No advanced controllers
 - No cloud-provider-specific features
-

Expected Outcome

By completing this lab, you should feel confident reading and writing basic Kubernetes manifests and understanding how Kubernetes interprets them.

Question 1

What are the **four required top-level fields** in every Kubernetes manifest?

Question 2

What is the role of the `spec` field in a Kubernetes manifest, and why is it considered the most important section?

Question 3

Create a **basic Deployment manifest** that:

- Uses `apps/v1`

- Is named `demo-app`
 - Defines **2 replicas**
-

Question 4

Extend the Deployment by adding:

- A container named `web`
 - Image `nginx`
 - Port `80`
-

Question 5

Update your Deployment to:

- Scale from **2 replicas to 4 replicas**
- Add a label `app: web`

Which fields changed? Which stayed the same?

Question 6

What is the purpose of **labels**, and how are they used by other Kubernetes resources?

Question 7

Create a **Service manifest** that:

- Exposes port `80`
 - Selects Pods using the label `app: web`
 - Uses type `ClusterIP`
-

Question 8

Explain how Kubernetes knows **which Pods** a Service should route traffic to.

Question 9

If one of the Pods crashes, do you need to update the YAML file?

Why or why not?

Question 10

In your own words, explain the difference between:

- **Declaring desired state**
 - **Executing commands imperatively**
-

Question 11

What happens if the `apiVersion` specified in a manifest not supported by the cluster?

Question 12

Why does a Deployment require both:

- a `selector`, and
- matching labels inside `template.metadata.labels`?

What could go wrong if they don't match?

Question 13

Which parts of a Kubernetes manifest are **user-defined**, and which parts are **managed by Kubernetes**?

(Hint: think about `spec` vs `status`.)

Question 14

What would happen if you manually delete a Pod that is managed by a Deployment?

Explain your answer using the concept of **reconciliation**.

Question 15

Why is YAML indentation critical in Kubernetes manifests?

What kind of issues can incorrect indentation cause?

Question 16

Where would you define **environment variables** for a container inside a Deployment manifest?

Question 17

What is the difference between:

- Changing a value in a YAML file, and
- Running a `kubectl` command that changes the same value?

How does this relate to declarative configuration?

Question 18

When troubleshooting a non-working application, why is checking the YAML manifest often the **first step**?
